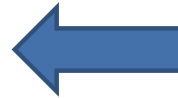


## 3- ARM CORTEX-M ARCHITECTURE

---

- History of ARM
- Cortex-M Features
- Cortex-M Instruction Set Architecture
- Instruction Set
- Memory Access
  - Memory-mapped I/O
- Exception Handling



## 3.3 CORTEX-M4 INSTRUCTION SET ARCHITECTURE

---

The Instruction Set Architecture (ISA) presents the Programmer's View of the processor, including:

- Data types
- Processor Modes
- Processor Registers
- Instruction Set
- Memory Accessing
- Exception Processing

## 3.3 CORTEX-M ISA

---

The Cortex-M4 instruction set can operate on the following data types:

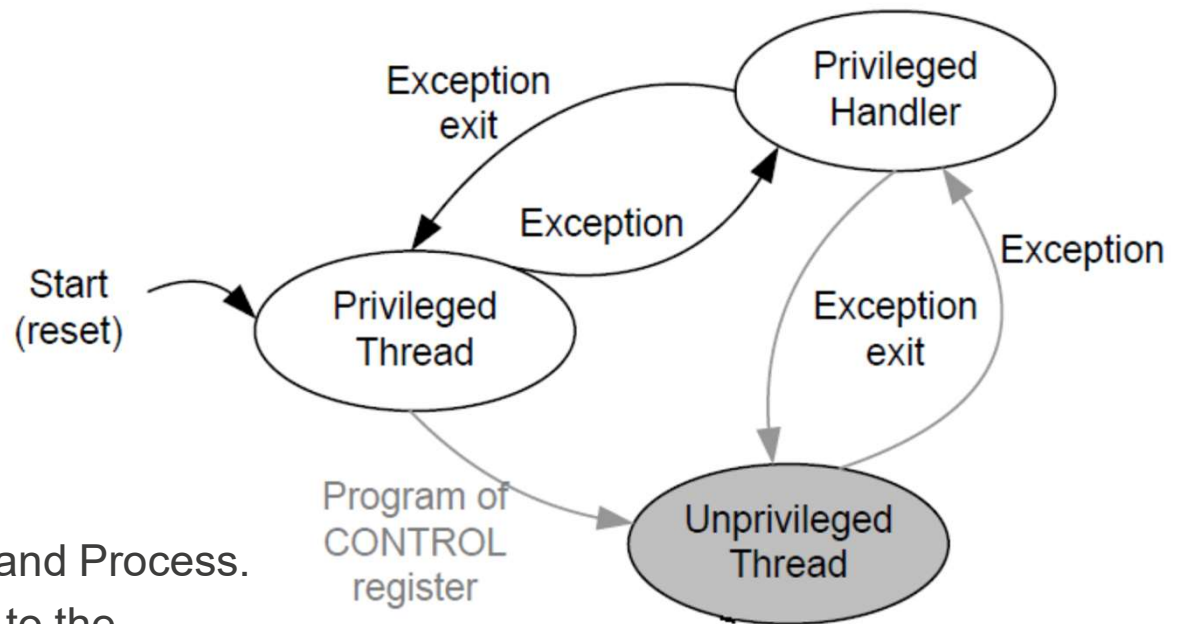
- **bit**: stores a single bit of information (0 or 1).  
Bit banding instructions can set or clear bits in specific memory regions.
- **byte**: 8-bit. Each byte in memory is individually addressable.
- **half-word**: 16-bit. The address of a half-word in memory is the address of its least significant byte.
- **word**: 32-bit. The address of a word in memory is the address of its least significant byte.
- **double-word**: 64-bit. Requires a register pair to be stored, such as R1:R0 (concatenation of R1 and R0 with R1 being the most significant word). The address of a double-word in memory is the address of its least significant byte.

## 3.3 CORTEX-M4 ISA

The Cortex-M4 processor modes are:

- Privileged Thread - mainly for OS execution
- Unprivileged Thread - mainly for Application code
- Privileged Handler - for exception handling code

Remark: there are two stacks: Main and Process.  
The usage of these stacks is related to the processor mode.



## 3.3 CORTEX-M4 ISA

---

The Cortex-M4 has two stacks:

- Main stack - addressed by MSP (Main Stack Pointer)  
mainly to be used by the Operating System (OS) and exception handlers.
- Process stack - addressed by PSP (Process Stack Pointer)  
mainly to be used by the Application threads.

Only one stack is active at any given time, as selected by the CONTROL register. Register R13 (SP) maps to the active stack pointer, either MSP or PSP.

## 3.3 CORTEX-M4 ISA

Table B1-1 Mode, privilege and stack relationship

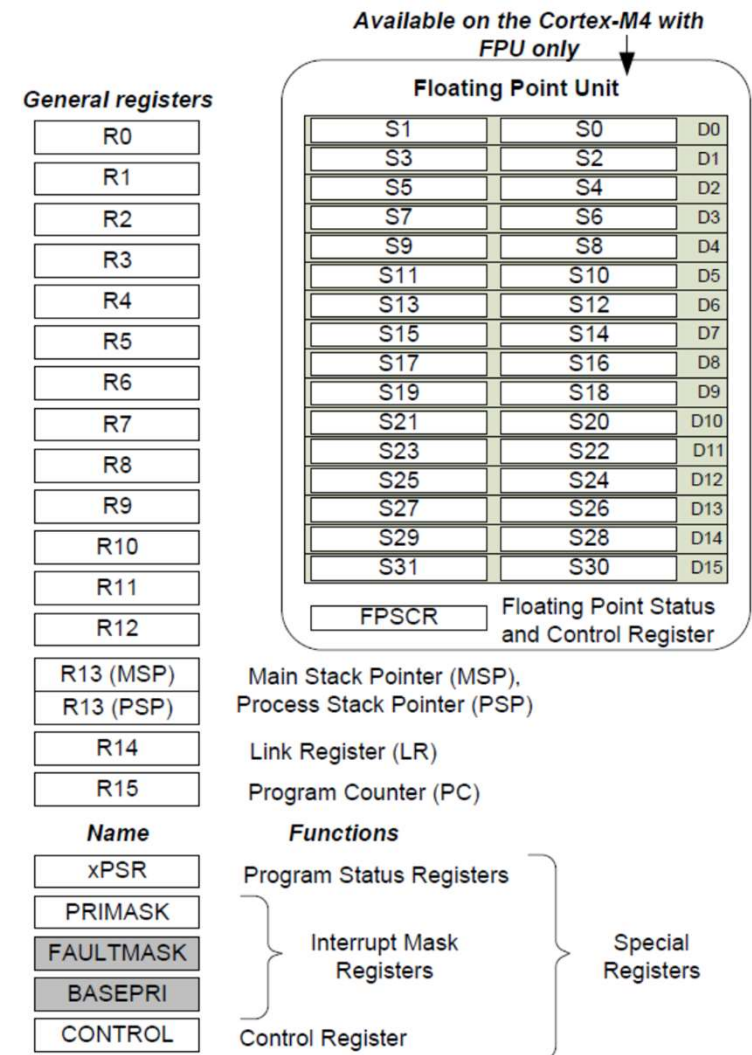
| Mode    | Privilege    | Stack pointer | Typical usage model                                                                                                                                                                                            |
|---------|--------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Handler | Privileged   | Main          | Exception handling.                                                                                                                                                                                            |
| Thread  | Privileged   | Main          | Execution of a privileged process or thread using a common stack in a system that only supports privileged access.                                                                                             |
|         |              | Process       | Execution of a privileged process or thread using a stack reserved for that process or thread in a system that only supports privileged access, or that supports a mix of privileged and unprivileged threads. |
| Thread  | Unprivileged | Main          | Execution of an unprivileged process or thread using a common stack in a system that supports privileged and unprivileged access.                                                                              |
|         |              | Process       | Execution of an unprivileged process or thread using a stack reserved for that process or thread in a system that supports privileged and unprivileged access.                                                 |

## 3.3 CORTEX-M4 ISA - REGISTERS

The register set of the Cortex-M4 consists of:

- General Purpose Registers (R0-R15)
- Floating Point Registers (S0-S31)
- Special Registers (xPSR, PRIMASK, FAULTMASK, BASEPRI, CONTROL)

All registers are 32-bit wide. Not all bits of the Special Registers are implemented.



## 3.3 CORTEX-M4 ISA - REGISTERS

---

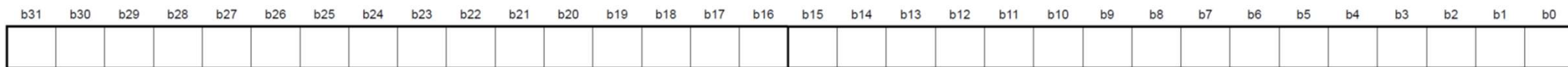
General Purpose Registers:

- 32-bit wide
- Low registers: R0 .. R7
- High registers: R8 .. R15
- Special usage:

R13 or SP is the Stack Pointer - points to the top of the stack

R14 or LR is the Link Register - stores the procedure return address

R15 or PC is the Program Counter - stores the address of the next instruction fetch



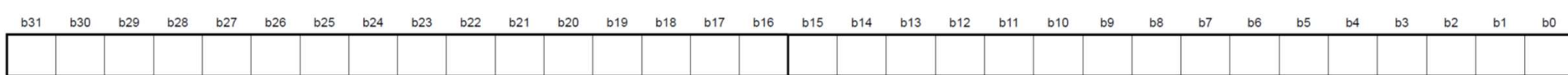


## 3.3 CORTEX-M4 ISA - REGISTERS

---

General Purpose Registers:

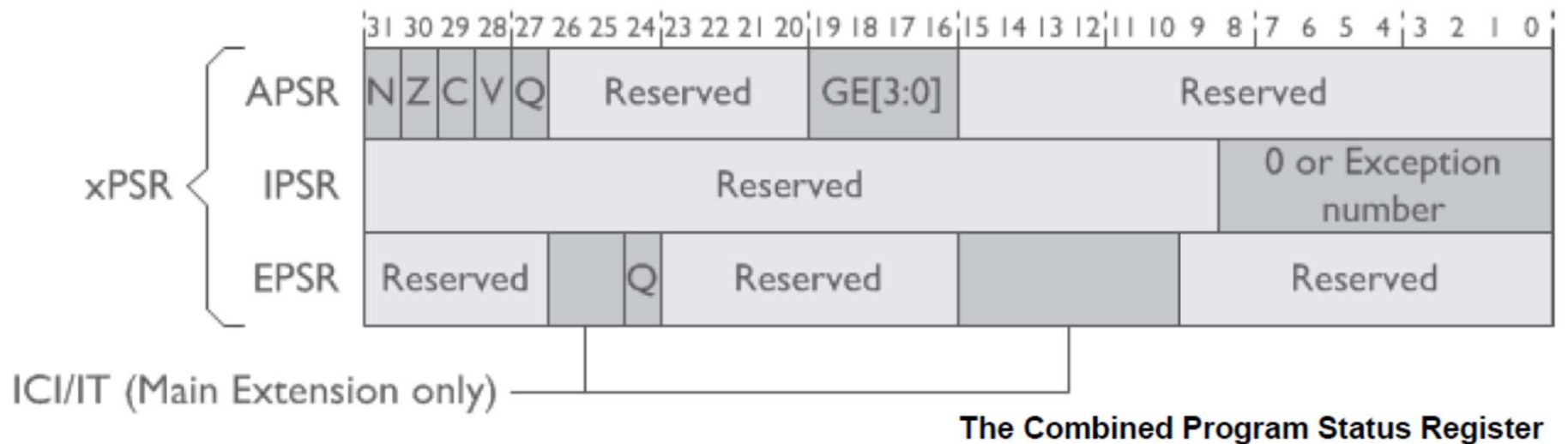
- the general-purpose registers are accessible by most instructions;
- b0 is the Least Significant Bit (LSb) and b31 is the Most Significant Bit (MSb);
- a register can hold an unsigned integer with values from 0 to 4,294,967,295 or a signed integer with values from -2,147,483,648 to 2,147,483,647;
- when using hexadecimal notation, a 32-bit register holds 8 hex digits. E.g. 0x1234 5678;
- Cortex-M architecture uses 32-bit memory addresses; hence, a single general-purpose register can hold a memory address.



## 3.3 CORTEX-M4 ISA - REGISTERS

### Special Purpose Registers - XPSR

- The three registers APSR, IPSR and EPSR can be accessed individually or combined as XPSR.



## 3.3 CORTEX-M4 ISA - XPSR

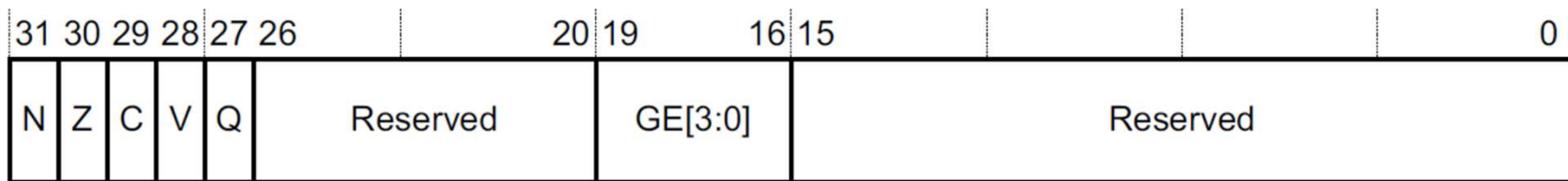
---

Mnemonics used to combine the XPSR component registers:

| Mnemonic | Registers accessed       |
|----------|--------------------------|
| IAPSR    | IPSR and APSR            |
| EAPSR    | EPSR and APSR            |
| XPSR     | All three xPSR registers |
| IEPSR    | IPSR and EPSR            |

### 3.3 CORTEX-M4 ISA - APSR

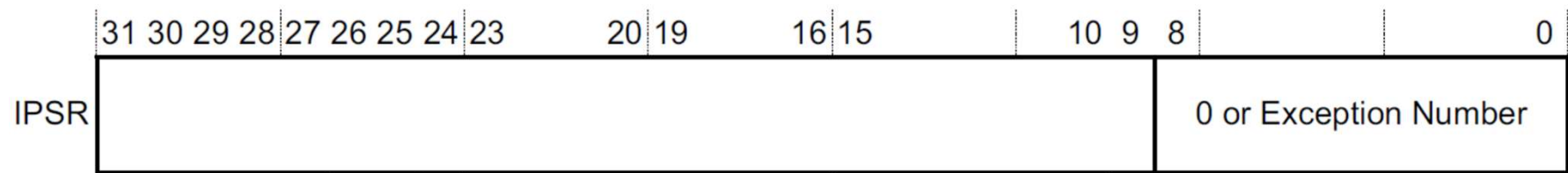
**APSR = Application Program Status Register**



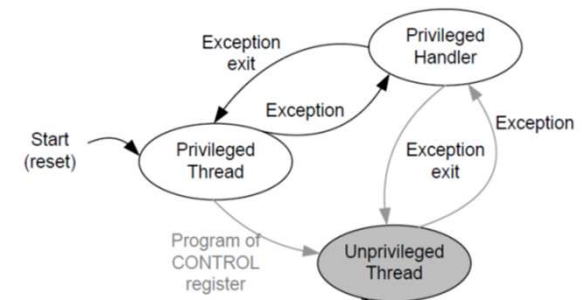
| Flag | bit   | Description                                                                                                                                      |
|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| N    | 31    | Negative. For two's complement results this bit is set to indicate that the result is negative.                                                  |
| Z    | 30    | Zero. This bit is set when the result is zero. After a comparison, this bit is set to indicate that the compared values are equal.               |
| C    | 29    | Carry. Set to indicate that the result of an unsigned addition produced overflow. Also set to indicate that an unsigned subtraction underflowed. |
| V    | 28    | Overflow. Set to indicate overflow in a signed arithmetic operation.                                                                             |
| Q    | 27    | Saturation bit. For DSP extension instructions.                                                                                                  |
| GE   | 19:16 | Greater than or Equal. For SIMD instructions.                                                                                                    |

## 3.3 CORTEX-M4 ISA - IPSR

IPSR = Interrupt Program Status Register

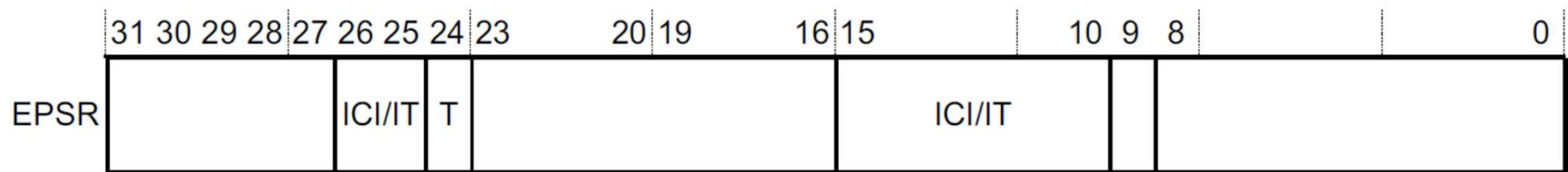


| Field            | bit   | Description                                                                                                                                                                                                                                                            |
|------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Exception Number | 8 : 0 | This field identifies the exception that is currently active (being serviced). The value 0 indicates that no exception is currently active.<br>If this value is non-zero, the processor is in Handler mode.<br>If this value is zero, the processor is in Thread mode. |



## 3.3 CORTEX-M4 ISA - EPSR

EPSR = Execution Program Status Register



| Field  | bit            | Description                                                                                                                                                                                                                                        |
|--------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T      | 24             | Thumb. This bit is set to indicate that the instruction set in use is Thumb.<br>On Cortex-M this bit must be set all the time, or an exception occurs.<br>On ARM7TDMI, Cortex-R and Cortex-A this bit is 0 when the ARM instruction set is in use. |
| ICI/IT | 26:25<br>15:10 | ICI - used for an interrupted exception-continuable multi-cycle load or store.<br>IT - provide context information for instructions in an IT block.                                                                                                |

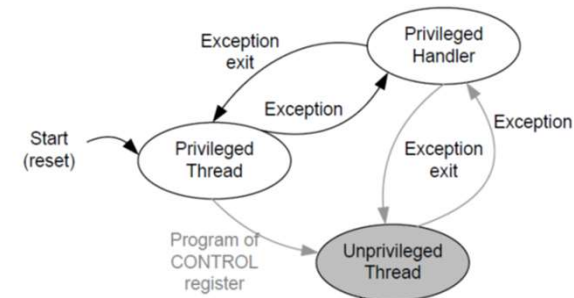
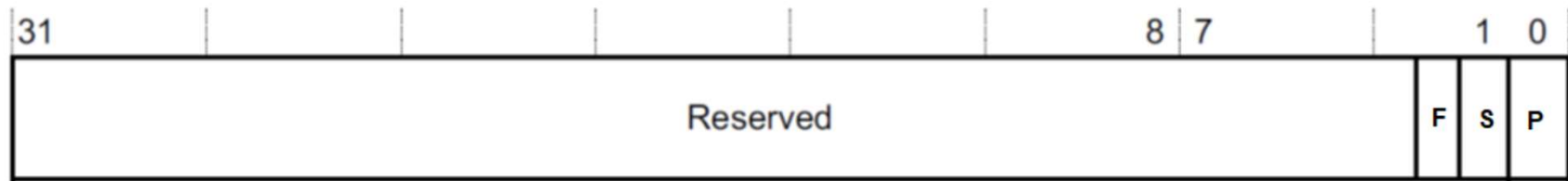
## 3.3 CORTEX-M4 ISA - SPECIAL REGISTERS



| Field   | bit          | Description                                                                                                                               |
|---------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| PM      | PRIMASK[0]   | Set to mask exceptions with configurable priority (priority 0 and lower). Reset to unmask.                                                |
| FM      | FAULTMASK[0] | Set to mask the HardFault exceptions and the configurable priorities (prio -1 and lower).                                                 |
| BASEPRI | BASEPRI[7:0] | Changes the priority level required for exception preemption. Affects only the currently executing code with lower priority than BASEPRI. |

## 3.3 CORTEX-M ISA - CONTROL REGISTER

### CONTROL Register



| Field | bit | Description                                                                                                                                                   |
|-------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nPRIV | 0   | When the processor is in Thread mode. 0 = privileged mode; 1 = unprivileged mode.                                                                             |
| SPSEL | 1   | Stack selection. 0 = use MSP (Main Stack); 1 = use PSP (Process Stack). In Handler mode this bit is always 0.                                                 |
| FPCA  | 2   | Implemented only when floating point is available.<br>0 = do not save floating point registers on exception;<br>1 = save floating point context on exception. |



## 3.3 CORTEX-M4 ISA - REGISTERS

---

Floating Point Registers:

The Cortex-M4 with optional floating-point extension, implements 32 32-bit floating point registers named S0 to S31.

These can be combined two by two forming 16 double precision (64-bit) floating point registers named D0 to D15. D0 is formed by S1:S0 (the concatenation of the registers S1 and S0 where S1 is the most significant word, i.e. the leftmost word).

# RESET

---

```
// TakeReset()
// =====

bit ExceptionActive[*];           // Conceptual array of 1-bit values for all exceptions
bits(32) vectortable = VTOR;
Mode CurrentMode;

TakeReset()
    R[0..12] = bits(32) UNKNOWN;
    SP_main = MemA[vectortable,4] & 0xFFFFFFF0;
    SP_process = ((bits(30) UNKNOWN):'00');
    LR = bits(32) UNKNOWN;         // Value must be initialised by software
    CurrentMode = Mode_Thread;
    APSR = bits(32) UNKNOWN;       // Flags UNPREDICTABLE from reset
    IPSR<5:0> = 0x0;               // Exception number cleared at reset
    PRIMASK.PM = '0';             // Priority mask cleared at reset
    CONTROL.SPSEL = '0';          // Current stack is Main
    CONTROL.nPRIV = '0';          // Thread is privileged
    ResetSCSRegs();               // Catch-all function for System Control Space reset
    ExceptionActive[*] = '0';     // All exceptions Inactive
    ClearEventRegister();          // See WFE instruction for more information
    start = MemA[vectortable+4,4]; // Load address of reset routine
    BLXWritePC(start);             // Start execution of reset routine
```